System Architecture StackUnderflow

Agile Software Engineering Capstone

Document field	Team entry
Project title	Financial Management System
Team name	StackUnderflow
Team members	Saw, Zane, Jeff, and Jaron
Student document owner	[Enter the person coordinating updates]
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	0.1 Draft, November 7th, 2026
Approval status	Draft

How to use this template

Use this document to explain the system's important structures, how its parts work together, and why the team made significant technical decisions. A teammate should be able to follow a user request through the system, find where data lives, and understand the main quality and security choices.

Replace each bracketed prompt with your team's information. Remove unused example rows and all remaining prompts before submission. Keep the document current as the project changes.

Create a thin first version during Sprint 0 or Sprint 1. Update it whenever the team changes a major component, interface, data model, deployment approach, security boundary, or important technical decision.

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
0.1	11/07/2026	Saw	Initial draft	[Reviewer]
[Version]	[Date]	[Name]	[What changed]	[Reviewer or decision]

Document outline

Section	Purpose
1 Architecture overview	Purpose, scope, status, and a short explanation of the system
2 Architecture drivers	Use cases, quality attributes, constraints, assumptions, and risks
3 Stakeholders and concerns	What different readers need from the architecture
4 Architecture at a glance	Selected architecture style, major elements, and rationale
5 System context view	Users, external systems, system boundary, and interactions
6 Logical view	Containers, components, responsibilities, and dependencies
7 Process and runtime view	Requests, processes, concurrency, state, and failure behavior
8 Development view	Source organization, modules, ownership, and build structure
9 Deployment view	Where software runs and how environments connect
10 Use case view	Scenarios that validate the other architecture views
11 Data architecture	Entities, relationships, ownership, validation, and migrations
12 Interfaces and APIs	Contracts, authentication, validation, errors, and versioning
13 Security and privacy	Trust boundaries, threats, secrets, identity, and data rules
14 Quality attributes	Performance, reliability, accessibility, maintainability, and testability
15 Architecture decisions	Short records of major decisions and consequences
16 Traceability and risks	Links to requirements, tests, prototypes, and unresolved concerns
17 Architecture evolution	How the team reviews and updates the architecture
18 Review and submission	Approvals and final completeness check

1 Architecture overview

1.1 Purpose and scope

What to write: *State what system this document covers, who should use it, and what is outside the architecture boundary.*

This document is the architecture of the StackUnderflow Financial Management System, a standalone web application designed to help users manage personal finances.

The system provides a dashboard for viewing financial information, managing budgets, tracking transactions, and reviewing financial summaries. It uses fabricated/hardcoded financial data for demonstration purposes and operates independently without connecting to real bank accounts.

The architecture covers the React frontend, ASP.NET Core Web API backend, SQLite database, and communication between these components. The system does not perform real banking transactions or payment processing.

Example: This document describes the web application, API, database, automated test pipeline, and demo deployment for Budget Buddy. It does not describe the internal design of third-party libraries or any real banking integration.

1.2 Architecture status

Item	Team entry
Architecture version	0.1 Draft, Date: November 7th, 2026
Current status	[Partly implemented] The frontend is implemented, and the backend is still in progress
Implementation baseline	Current Capstone Prototype Demo
Known gaps	Authentication, password hashing, API integration, testing, and deployment

1.3 Short architecture narrative

What to write: *Explain the system in one or two paragraphs using the same names that appear in the diagrams.*

This web application uses a React frontend, an ASP.NET Core Web API backend, and an SQLite database. The frontend provides the user interface, including the dashboard, budget pages, financial summaries, and charts. The frontend communicates with the backend through API requests. The backend is responsible for processing application requests and handling data access through the database. SQLite stores application data. The application uses fabricated financial information and is being developed as a prototype for demonstration purposes.

Example: Budget Buddy uses a React web client, a REST API, and PostgreSQL. The browser communicates with the API over HTTPS and never connects directly to the database. The API authenticates the user, validates each request, applies business rules, and limits every user-owned query to the authenticated user. A continuous integration pipeline runs unit and integration tests before the team merges code.

---

2 Architecture drivers

Architecture drivers are requirements or risks that strongly influence the system structure. Focus on the few items that change architectural choices.

2.1 Key use cases

Use case ID	Use case	Why it affects architecture	Related view or decision
UC-01	Register and sign in	Requires identity storage, password hashing, sessions, and authorization	Security, data, and runtime views
UC-02	Create and review a monthly budget	Requires user-owned data and calculated summaries	Logical and data views
UC-03	View the financial dashboard	Requires financial summaries and chart data	Logical and runtime views
UC-04	View and track transactions	Requires transaction records and categorization of expenses such as utility bills, rent, food, etc.	Logical and data views

2.2 Quality attributes

Quality ID	Quality scenario	Measure or response	Architecture effect
QA-01	When a signed-in user requests a budget, the system must return only that user's data	Authorization tests pass for positive and negative cases	Every user-owned query uses authenticated user scope
QA-02	The dashboard should load under normal demo conditions	95 percent of test requests complete within [target]	Indexes, compact API responses, and measured queries
QA-03	Application handles user input.	Invalid information should be rejected	Input validation
QA-04	Application is easy to maintain	Fronten and backend remain separated	Component-based frontend and API-based backend.
[ID]	[Trigger, condition, and expected response]	[Measurable target]	[Design tactic]

2.3 Constraints assumptions and architecture risks

Type	Statement	Source	Architecture effect	Owner or validation
Constraint	Use only fabricated financial and card data	Sponsor brief	No real banking connection or payment processing	Whole team
Assumption	Example: the demo environment supports the selected database	Hosting documentation	Deployment design depends on managed database availability.	DevOps owner checks in Sprint 1
Constraint	The project is developed as a capstone prototype	CS-451	The focus on core functionality and demonstration	Whole team
Constraint	Reach, ASP.Net core Web API, and SQLite are used	Project Documentation, Requirement Documentation	Determines the technology structure	Whole team
Assumption	The application runs in a local development environment	Team brief	Local frontend, backend, and database configuration	Whole team
Risk	Authentication is not fully implemented.	Team brief	User identity and account protection remain incomplete.	Whole team
Risk	Some API and database functionality is still under development	Team brief	Additional integration and testing are required.	Saw, and Jeff
Risk	[Technical uncertainty]	[Source]	[Possible effect]	[Owner and validation plan]

3 Stakeholders and concerns

Stakeholder	Main concerns	Architecture information needed	View or evidence
Users	Usability, privacy, correct behavior	User journeys and visible system behavior	Use case and logical views
Developers	Clear responsibilities and interfaces	Components, APIs, dependencies, and source structure	Logical and development views
Testers	Testable boundaries and failure behavior	Interfaces, quality scenarios, environments, and logs	Runtime, deployment, and quality sections
Sponsor or instructor	Scope, feasibility, risk, and evidence	Context, major decisions, tradeoffs, and status	Overview, drivers, ADRs, and traceability
Users	Usability, financial information, privacy	Dashboard, budget and transaction functionality	Runtime and logical views
Developers	Code organization and integration	API responses, validation and test cases	Runtime, deployment and quality sections
Sponsor/client	Application urpose and feasibility	System scope and financial management functionality	Logical and development views.

Stakeholder	Main concerns	Architecture information needed	View or evidence
[Stakeholder]	[Concern]	[Needed information]	[View]

4 Architecture at a glance

4.1 Selected architecture style

The system follows a client-server architecture with three layers: a React frontend for the user interface, an ASP.NET Core Web API for application logic, and SQLite for data storage. This separation allows the team to develop and maintain the frontend and backend independently.

The current frontend uses sample data and local state. Backend integration and authentication are still in progress.

4.2 Major architecture elements

Element	Responsibility	Important interface	Data owned or used	Team owner
Web client	Renders pages, performs accessible client validation, and calls the API	HTTPS JSON	Temporary UI state	Front-end lead
API	Authenticates, validates, authorizes, applies rules, and returns structured errors	REST endpoints	Users, budgets, and transaction data through repository	Backend lead
Database	Stores persistent application data	SQL through repository layer	Users, profiles, budgets, categories, and fabricated transactions	Data owner
Frontend	Displays dashboard, budgets and financial information	REACT	Users, profiles, budgets, categories, and fabricated transactions	Whole Team
Backend API	Handles application requests and business logic	SQLite (Maybe Postman for test cases)	Users, profiles, budgets, categories, and fabricated transactions	Saw, Jeff, Zane
Charts	Visualizes financial information	Recharts	Users, profiles, budgets, categories, and fabricated transactions	Whole Team
[Element]	[Responsibility]	[Interface]	[Data]	[Owner]

5 System context view

5.1 Context diagram

What to write: /

```
None
flowchart LR
    U["User"] -->|"Enters form information and requests financial views"| S["Financial Management System"]
    S -->|"Displays validation messages, budget results, account information, and transactions"| U
```

The Financial Management System is the system boundary. It includes the user interface and the planned backend and database. The user interacts with the system through a web browser. No external banking, payment, email, or AI service is currently connected.

5.2 Context explanation

Actor or external system	Interaction with the system	Information exchanged	Trust or dependency concern
Registered user	Uses browser to manage budget	Credentials, profile, budget, and fabricated transaction data	User should access only their own data
User accessing login and registration	Enters login information, completes registration fields, or requests password recovery.	Name, email address, password fields, and validation messages.	These are currently frontend demonstrations. Authentication, account creation, and password-reset processing require backend integration.
User managing finances	Enters monthly income and expenses, reviews savings and wishlist accounts, and views dashboard information.	Budget amounts, calculated totals, savings goals, balances, and chart data.	The completed backend must validate submitted information and restrict access to the user's own records.
User reviewing transaction history	Selects a month and switches between recent and all transactions.	Transaction dates, descriptions, account names, amounts, and balances after transactions.	Records currently come from fabricated frontend data rather than a connected database.

Commerce Bank is the project sponsor, not a connected banking service. The AI assistant is currently a visual chat interface without an external AI connection.

6 Logical view

6.1 Container diagram

```
None
flowchart LR
    U["User"] -->|"Browser interaction"| F

    subgraph S["Financial Management System"]
        F["React Frontend"]
        A["ASP.NET Core Web API"]
        D[("SQLite Database")]
        F -.->|"Planned HTTP / JSON communication"| A
        A -.->|"Planned database access"| D
    end
```

Solid arrows represent current interaction; dashed arrows represent intended integration.

The React frontend provides pages, navigation, input validation, budget calculations, and charts. The intended ASP.NET Core Web API handles authentication, business rules, and data access. SQLite provides persistent storage.

The reviewed frontend currently uses local state and fabricated data without API integration. Backend implementation progress must be confirmed with the backend team.

6.2 Component view

```
None
flowchart TD
    A["App.js / React Router"] --> L["Authentication pages"]
    A --> D["Dashboard"]
    A --> AC["Accounts"]
    A --> B["Budget"]
```

```
A --> F["Financial Summary"]

D --> H["Shared AppHeader"]
AC --> H
B --> H
F --> H
D --> C["Recharts"]
AC --> N["AccountHeader / AccountNode"]
F --> AI["AI chat interface"]
```

Component	Responsibility	Depends on	Provided interface	Requirement or use case
Budget service	Applies budget and category rules	Budget repository and authenticated user context	Service methods used by API routes	UC-02
Authentication pages	Provide login, registration, password confirmation, and password-recovery demonstrations.	React Router, React Bootstrap, and AuthLayout for registration and recovery.	Authentication forms.	UC-01: Register and sign in.
Dashboard	Displays financial totals, account information, activity tables, and charts.	AppHeader, Recharts, and sample data.	Dashboard views.	UC-03: View the financial dashboard.
Accounts	Displays emergency-savings progress and savings/wishlist cards.	AppHeader, AccountHeader, and AccountNode.	Account views and demonstration states.	Savings and wishlist management.
Budget	Validates income and expenses, calculates remaining money, and supports editing and resetting reports.	React state, React Bootstrap, and AppHeader.	Budget form and summary.	UC-02: Create and review a monthly budget.
Financial Summary	Displays transactions, filters by month, and shows balances after transactions.	React state, AppHeader, and sample transaction data.	Transaction-history table.	UC-04: View and track transactions.

Component	Responsibility	Depends on	Provided interface	Requirement or use case
AI chat interface	Opens and closes a chat window; messaging is not implemented.	Financial Summary state.	Chat button and window.	Planned savings feedback.
AppHeader	Provides shared navigation between pages.	React Router and React Bootstrap.	Navigation bar.	Supports all main pages.

The current pages use separate local state and sample datasets. Budget submissions do not yet update the dashboard or transaction history through a shared backend.

7 Process and runtime view

7.1 Key request or event flows

What to write: *Use a sequence diagram or numbered flow for important runtime behavior. Show validation, authentication, authorization, data access, errors, and responses.*

Example: Create budget flow: 1. Web client sends an authenticated POST request. 2. API validates the request body. 3. Authorization obtains the user ID from the session. 4. Budget service applies business rules. 5. Repository inserts the budget using the user ID. 6. API returns the created budget or a structured error.

[Insert sequence diagram or numbered flow here]

7.2 Processes concurrency state and failure

Concern	Design decision	Failure behavior	How the team will test or observe it
Session state	Example: server-managed session with secure cookie	Expired session returns 401 and client redirects to login	Integration test and structured log
Concurrent update	[How simultaneous changes are handled]	[Expected error or retry]	[Test or monitoring]
External dependency	[Timeout, retry, or fallback rule]	[Failure response]	[Verification]

8 Development view

8.1 Source and module organization

What to write: *Show how the architecture maps to repositories, folders, packages, modules, and development ownership. Keep names consistent with the logical view.*

Repository or module	Purpose	Main dependencies	Build or test command	Owner
web	Front-end application	API contract and component library	[Command]	Front-end lead
api	Routes, services, repositories, and authentication	Database and session library	[Command]	Backend lead
[Module]	[Purpose]	[Dependencies]	[Command]	[Owner]

8.2 Team and integration structure

What to write: *Explain how team ownership, pull requests, shared interfaces, and integration practices support the architecture. Note any Conway's Law risk where team boundaries could create unwanted system boundaries.*

---

9 Deployment view

9.1 Deployment diagram

What to write: *Show development, test, and demo or production environments. Include browser, hosting, API process, database, network paths, configuration, and secrets sources.*

[Insert deployment diagram here]

9.2 Environment inventory

Environment	Purpose	Deployed elements	Data	Configuration and secrets	Owner
Local development	Build and unit test	Web, API, local database	Fabricated developer data	Local environment file excluded from source control	Each developer
Test or demo	Integration testing and reviews	[Elements]	Fabricated shared data	[Secret store and config source]	[Name]
[Environment]	[Purpose]	[Elements]	[Data]	[Configuration]	[Owner]

10 Use case view

The use case view is the plus one view. It shows how the logical, runtime, development, and deployment views work together for an important scenario.

Scenario	Logical elements	Runtime flow	Deployment nodes	Evidence
Register and sign in	Web forms, auth route, auth service, user repository	Validate, hash, store, create session, return result	Browser, API host, database	Sequence diagram and automated tests
[Use case]	[Components]	[Flow summary]	[Nodes]	[Test, prototype, or demo]

11 Data architecture

11.1 Data model

What to write: *Insert an entity relationship diagram or class model. Show key fields, primary and foreign keys, relationships, required values, and ownership boundaries.*

[Insert data model diagram here]

11.2 Data dictionary and ownership

Entity or data set	Purpose	Important fields	Owner or access rule	Validation retention or migration
User	Stores identity and profile reference	ID, email, password hash, created date	API only; user can access own profile	Unique email; password never stored in plain text
Budget	Stores a user's monthly spending plan	ID, user ID, month, category amounts	Queries must use authenticated user ID	Positive amounts; schema migrations tracked
[Entity]	[Purpose]	[Fields]	[Ownership]	[Rules]

12 Interfaces and APIs

12.1 Interface catalog

Interface ID	Provider and consumer	Purpose	Protocol or format	Authentication	Version or status
API-01	Web client to API	Create a budget	POST /api/budgets with JSON	Required session	v1 proposed

Interface ID	Provider and consumer	Purpose	Protocol or format	Authentication	Version or status
[ID]	[Provider and consumer]	[Purpose]	[Contract]	[Rule]	[Version or status]

12.2 Contract details

Contract field	Team entry
Endpoint or interface	[Name and method]
Purpose	[Behavior provided]
Request example	[Representative request]
Success response	[Status and response]
Validation errors	[Status, code, and message structure]
Authentication and authorization	[Required identity and ownership checks]
Compatibility or versioning	[How changes are handled]

13 Security and privacy

13.1 Trust boundaries and threats

What to write: *Mark trust boundaries on the context or deployment diagram. Identify likely misuse and failure cases without listing generic security terms.*

Threat or misuse case	Affected asset	Architecture control	Verification evidence	Owner
User requests another user's budget by changing an ID	Private budget data	API scopes every query to the authenticated user ID	Negative authorization integration test	Backend lead
Malicious input reaches database query	Database integrity	Schema validation and parameterized repository queries	Unit and integration tests	Data owner
[Threat]	[Asset]	[Control]	[Test or review]	[Owner]

13.2 Security and privacy decisions

Area	Team decision
Authentication	[Library or service, session or token approach, logout, and expiration]
Password handling	[Supported hashing library and no plain-text storage or logging]
Authorization	[Roles and user-owned resource checks]
Input and output protection	[Validation, encoding, parameterized access, file rules]

Area	Team decision
Secrets	[Storage, rotation, and access]
Logging	[Events recorded and sensitive values excluded]
Financial data	[Fabricated data rule and prohibited real data]

14 Quality attributes

Quality attribute	Target or scenario	Architecture tactic	Evidence and owner
Performance	[Measured response target under stated load]	[Caching, indexing, compact data, or other tactic]	[Test and owner]
Reliability	[Expected behavior when a component fails]	[Timeout, transaction, recovery, or validation tactic]	[Test and owner]
Accessibility	[Relevant conformance or user task]	[Semantic UI, keyboard support, contrast, and testing]	[Audit and owner]
Maintainability	[Change scenario]	[Clear boundaries, interfaces, and code ownership]	[Review and owner]
Testability	[Behavior that must be isolated or observed]	[Dependency boundaries, test data, logs, and automated checks]	[Pipeline evidence]
Observability	[Question the team must answer during failure]	[Structured logs, error IDs, and health checks]	[Demo or test]

15 Architecture decisions

Create a short Architecture Decision Record for every decision that would be expensive or disruptive to change later. Keep rejected alternatives so future teammates understand the reasoning.

15.1 Decision index

ADR ID	Decision title	Status	Date	Owner	Related requirements
ADR-001	Example: Use server-managed authentication sessions	Accepted	[Date]	[Name]	FR-01, NFR-SEC-01
ADR-__	[Decision title]	[Proposed / Accepted / Replaced]	[Date]	[Name]	[IDs]

15.2 Architecture Decision Record template

Field	Content
Decision	[What the team decided]

Field	Content
Status and date	[Proposed / Accepted / Replaced and date]
Context	[Problem, requirement, constraint, and forces]
Options considered	[Realistic alternatives]
Choice and reason	[Selected option and evidence-based rationale]
Consequences	[Benefits, costs, risks, and follow-up work]
Owner and reviewers	[Names or roles]
Related evidence	[Prototype, test, requirement, issue, or source]

16 Traceability and risks

16.1 Architecture traceability

Requirement or quality ID	Architecture element or decision	Verification evidence	Implementation status
FR-01 Registration	Auth route, auth service, user repository, ADR-001	Unit and integration tests	[Planned / Active / Implemented]
[ID]	[View, component, or ADR]	[Test, prototype, review, or demo]	[Status]

16.2 Architecture risks and open questions

ID	Risk or question	Why it matters	Validation or response	Owner	Target date	Status
AR-01	Can the selected hosting plan run the database and API within limits?	Deployment choice may fail late	Deploy a thin end-to-end slice in Sprint 1	DevOps owner	[Date]	Open
AR-__	[Risk or question]	[Effect]	[Prototype, research, or response]	[Name]	[Date]	[Open / Closed]

17 Architecture evolution

What to write: *Explain who maintains this document, when the team reviews it, how proposed and implemented elements are marked, and what events require an update.*

Example: The architecture lead reviews this document during backlog refinement once per sprint. Component owners update diagrams and interfaces when implementation changes. The team records significant decisions in ADRs and updates traceability after the related tests pass.

17.1 Diagram rules

[ ] Give each diagram a title, version or date, purpose, and legend when needed.

[ ] Label arrows with the interaction or protocol, such as HTTPS JSON request or SQL query.

[ ] Use the same component names in diagrams, source folders, interfaces, and tables.

[ ] Use one readable diagram for one main question instead of one giant diagram.

[ ] Mark proposed and implemented elements clearly and update their status each sprint.

[ ] Do not use screenshots of code as architecture diagrams.

18 Review and submission

18.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
[Name]	Project sponsor or mentor	[Approve / Revise]	[Comments]	[Date]
Shah, Syed Jawad Hussain	Instructor	[Approve / Revise]	[Comments]	[Date]
[Name]	Architecture lead	[Team acceptance]	[Comments]	[Date]

18.2 Final submission checklist

[ ] Architecture drivers come from real use cases, nonfunctional requirements, constraints, and risks.

[ ] The document includes context, logical, runtime, development, deployment, and use case views as needed.

[ ] Diagrams use readable labels and names that agree with the implementation.

[ ] The data model protects ownership boundaries and prohibits real financial or card data.

[ ] Interfaces describe authentication, authorization, validation, success, and error behavior.

[ ] Security, secrets, logging, accessibility, reliability, performance, maintainability, and testability are addressed.

[ ] Major decisions include alternatives, rationale, consequences, owner, and date.

[ ] Architecture claims link to requirements and verification evidence.

[ ] Proposed and implemented elements are clearly distinguished.

[ ] All bracketed prompts and unused example rows have been removed.

[ ] The version, date, author list, diagram dates, and review decisions are current.

Recommended filename: TeamName_SystemArchitecture_v1.0.docx